oop大作业分析

姓名：徐锦云

学号：23307130447

专业：信息安全

目录

oop大作业分析
- 目录
- 新增或替换的资源文件和源代码
- 背景介绍
  - 配置环境
    - 下载预编译 SDK
    - 配置头文件路径
- 算法解释
  - 代码核心部分的具体说明
    - 各代码模块核心逻辑与作用
      - **1. Fruit（Fruit.cpp/h）**
      - **2. Game（Game.cpp/h）**
      - **3. GameOverScreen（GameOverScreen.cpp/h）**
      - **4. GameScreen（GameScreen.cpp/h）**
      - **5. Grid（Grid.cpp/h）**
      - **6. MenuScreen（MenuScreen.cpp/h）**
      - **7. Screen（Screen.cpp/h）**
      - **8. Snake（Snake.cpp/h）**
      - **9. SnakeNode（SnakeNode.cpp/h）**
      - **10. main.cpp**
    - 模块关联结构图
  - 实验文档中五个要求的实现（具体代码）
    - 贪食蛇的控制
    - 水果的控制
    - 绘制精灵版本的贪食蛇
    - 整体界面的修改
    - 理清代码
  - 漏洞修复
  - 程序运行逻辑改进
  - 视听效果修改
  - 其他的可行性修改
    - 关于水果的分布
    - 关于蛇的身体构造
    - 关于边界问题
  - 对于正规贪吃蛇游戏的一些分析以及对于本项目的启发
    - 解密过程
    - 正式开始分析
      - **1. 模块化与动态加载**
      - **2. 现代语法转译支持**
      - **3. 兼容性处理**
      - **4. 依赖管理**
      - **5. 代码组织**

效果展示
关于cmake
执行
运行界面
视频

新增或替换的资源文件和源代码

新增pics文件夹（保存了snakebody.jpg和snakehead.jpg两个图片文件，用于绘制精灵）

*关于snakehead的说明：为了独特性，选取了一块蛋糕作为蛇的头（抠图网站：https://www.remove.bg/zh/upload），为了节省空间，对原图进行了压缩（压缩网站：https://tinypng.com/）

源代码新增：Grid.h，Grid.cpp

新增pickup.wav，用来展示捡到水果之后的声音

替换bg_music.ogg，变成After_All-boomboombeckett.mp3

别的文件主要做了代码修改，没有文件数量上的更新。

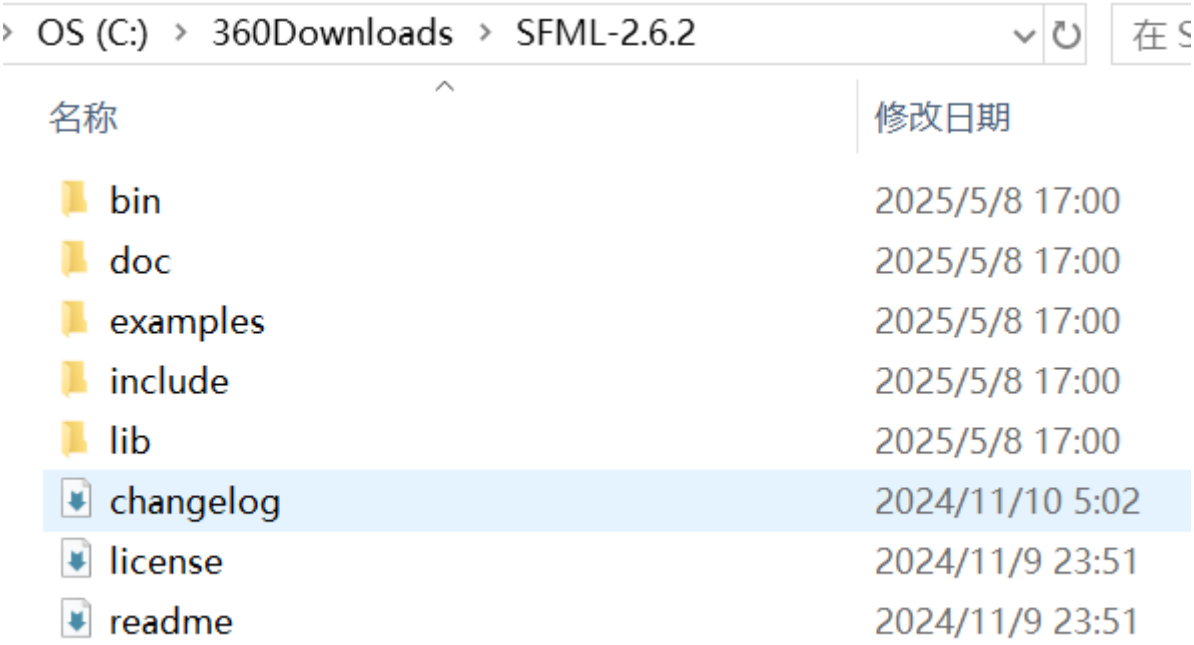
背景介绍

利用SFML或者SDL库，利用C++开发具有界面(GUI)的游戏：贪吃蛇游戏，同时需要dll链接。

配置环境

下载预编译 SDK

- 访问 SFML 官网，选择与 Visual Studio 版本匹配的预编译包。
- 解压下载的压缩包到任意目录



配置头文件路径

1. 打开 VS Code，进入项目文件夹。
2. 按下 `Ctrl+Shift+P` ，输入 `C/C++: Edit Configurations (UI)` ，选择后会自动生成 `c_cpp_properties.json` 。

3. 在 `IncludePath` 中添加 SFML 的头文件路径：

```
{
    "configurations": [
        {
            "name": "Win32",
            "includePath": [
                "${workspaceFolder}/**",
                "C:\\360Downloads\\SFML-2.6.2\\include",
                "C:\\MinGW\\include"
            ],
            "defines": [
                "_DEBUG",
                "UNICODE",
                "_UNICODE"
            ],
            "windowsSdkVersion": "10.0.22621.0",
            "compilerPath": "cl.exe",
            "cStandard": "c17",
            "cppStandard": "c++17",
            "intelliSenseMode": "windows-msvc-x64"
        }
    ],
    "version": 4
}
```

配置好之后，项目会自动生成settings.json。到此为止，我们配置好了所有环境，接下来就是代码逻辑的问题了，我会在下文对修改做出具体说明。

算法解释

使用文字描述和绘制核心部分的流程图(15);截图重要代码及注释，并解释其实现逻辑(10);创新代码及说明(10)。

代码核心部分的具体说明

各代码模块核心逻辑与作用

---

1. Fruit（Fruit.cpp/h）

- **作用**：表示游戏中的水果对象。
- **逻辑**：
  - 存储位置、颜色和半径属性。
  - 提供渲染方法和获取颜色/位置接口。
  - 被蛇碰撞时触发生长逻辑。

---

2. Game（Game.cpp/h）

- **作用**：游戏主循环与全局管理。
- **逻辑**：
  - 管理窗口、背景音乐和背景颜色。
  - 处理全局输入（如切换背景色）。

- 控制当前活动界面（`Screen`）的更新与渲染。

---

3. GameOverScreen（GameOverScreen.cpp/h）

- **作用**：游戏结束界面。
- **逻辑**：
  - 显示最终得分和操作提示（重玩/退出）。
  - 通过按键切换回游戏或关闭窗口。

---

4. GameScreen（GameScreen.cpp/h）

- **作用**：游戏主界面逻辑。
- **逻辑**：
  - 管理蛇（`Snake`）、水果（`Fruit`）和网格（`Grid`）。
  - 处理用户输入（显示网格、切换网格颜色）。
  - 生成随机位置和颜色的水果。

---

5. Grid（Grid.cpp/h）

- **作用**：绘制游戏背景网格。
- **逻辑**：
  - 根据单元格大小生成网格线。
  - 支持动态切换网格颜色。

---

6. MenuScreen（MenuScreen.cpp/h）

- **作用**：游戏主菜单界面。
- **逻辑**：
  - 显示标题和操作提示（开始游戏/退出）。
  - 实现标题旋转动画。

---

7. Screen（Screen.cpp/h）

- **作用**：界面抽象基类。
- **逻辑**：
  - 定义通用接口（`handleInput`、`update`、`render`）。
  - 派生类包括 `MenuScreen`、`GameScreen`、`GameOverScreen`。

---

8. Snake（Snake.cpp/h）

- **作用**：蛇的移动与碰撞逻辑。
- **逻辑**：
  - 支持键盘和鼠标控制方向。
  - 检测边界碰撞和自碰撞。
  - 根据水果颜色触发不同生长长度。

9. SnakeNode（SnakeNode.cpp/h）

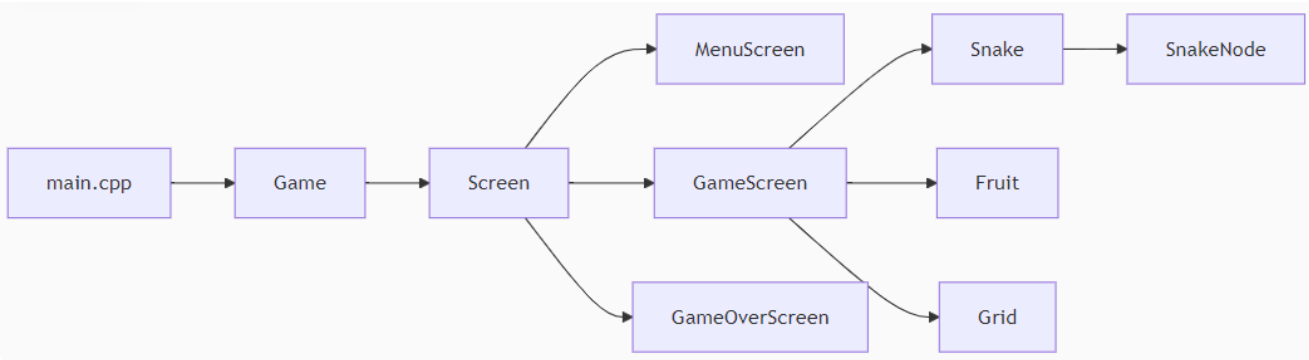
- **作用**：蛇的节点渲染与位置管理。
- **逻辑**：
  - 使用图片资源渲染蛇头和身体。
  - 根据方向动态调整旋转角度。

10. main.cpp

- **作用**：程序入口。
- **逻辑**：初始化 `Game` 对象并启动游戏循环。

模块关联结构图

```
main.cpp
└── Game (Game.cpp/h)
    ├── Screen (Screen.cpp/h)
    |   ├── MenuScreen (MenuScreen.cpp/h)
    |   ├── GameScreen (GameScreen.cpp/h)
    |   └── GameOverScreen (GameOverScreen.cpp/h)
    ├── Snake (Snake.cpp/h)
    |   └── SnakeNode (SnakeNode.cpp/h)
    ├── Fruit (Fruit.cpp/h)
    └── Grid (Grid.cpp/h)
```



- **箭头含义**：模块调用或包含关系。
- **关键交互**：
  - `Game` 管理当前 `Screen` ，通过 `GameScreen` 控制 `Snake` 和 `Fruit` 。
  - `Snake` 依赖 `SnakeNode` 实现节点渲染。
  - `GameScreen` 调用 `Grid` 绘制网格，并与 `Snake` 交互检测碰撞。

实验文档中五个要求的实现（具体代码）

贪食蛇的控制

源代码只支持4个方向的运行，增加可以通过鼠标控制贪食蛇的运动。当按下鼠标键时，设置一个方向向量，该方向向量为鼠标所在位置(MousePosition)与蛇头所在位置(SnakePosition)的差值。下一时刻，贪食蛇按照该向量的方向运动；运动的距离为1个标准单位。

具体改动：

在enum class Direction中增加了Any，表示此时由鼠标控制方向，方向可能是任意的。

```
namespace sfSnake
{
    enum class Direction
    {
        /*new*/
        Left, Right, Up, Down, Any
    };
```

在Snake类中新增了directionVector_，用于记录鼠标控制方向时觅食蛇的前进方向单位向量。

```
/*new*/
sf::Vector2f directionVector_;
```

在Snake类中新增了getDirectionVector()成员函数，用于获取属性directionVector_。

```
/*new*/
sf::Vector2f getDirectionVector() const;
```

重写了Snake::handleInput()函数，使其能处理鼠标左键按下时的情况：先获取鼠标相对于窗口的相对位置，如果鼠标位置在窗口内，则计算鼠标位置与当前蛇头位置的差值向量，并将其化为单位向量赋给directionVector，并将direction的状态置为Any。（如果差值向量的长度为0，说明鼠标位置与蛇头位置重合，则什么也不做）

```
void Snake::handleInput(sf::RenderWindow& window)
{
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::Up) && direction_ != Direction::Down)
        direction_ = Direction::Up;
    else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Down) && direction_ != Direction::Up)
        direction_ = Direction::Down;
    else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Left) && direction_ != Direction::Right)
        direction_ = Direction::Left;
    else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Right) && direction_ != Direction::Left)
        direction_ = Direction::Right;
    else if (sf::Mouse::isButtonPressed(sf::Mouse::Left))
    {
        auto mousePositioni = sf::Mouse::getPosition(window);
        sf::Vector2f mousePosition(static_cast<float>(mousePositioni.x), static_cast<float>(mousePositioni.y))
        auto windowSize = window.getSize();

        if (mousePosition.x >= 0 && mousePosition.x < windowSize.x &&
            mousePosition.y >= 0 && mousePosition.y < windowSize.y)
        {
            auto snakePosition = nodes_[0].getPosition();

            sf::Vector2f difference = mousePosition - snakePosition;

            float length = std::sqrt(std::pow(difference.x, 2) + std::pow(difference.y, 2));

            if (length > 0.0f)
            {
                directionVector_ = difference / length;
                direction_ = Direction::Any;
            }
        }
    }
}
```

为Snake::grow()添加了direction_的状态为Any的情况。

```
void Snake::grow(unsigned count)
{
    /*new*/
    for (unsigned i = 0; i < count; ++i){
        switch (direction_)
        {
        case Direction::Up:
            nodes_.push_back(SnakeNode(sf::Vector2f(nodes_[nodes_.size() - 1].getPosition().x,
                nodes_[nodes_.size() - 1].getPosition().y + 2 * SnakeNode::Radius)));
            break;
        case Direction::Down:
            nodes_.push_back(SnakeNode(sf::Vector2f(nodes_[nodes_.size() - 1].getPosition().x,
                nodes_[nodes_.size() - 1].getPosition().y - 2 * SnakeNode::Radius)));
            break;
        case Direction::Left:
            nodes_.push_back(SnakeNode(sf::Vector2f(nodes_[nodes_.size() - 1].getPosition().x + 2 * SnakeNode::Radius,
                nodes_[nodes_.size() - 1].getPosition().y)));
            break;
        case Direction::Right:
            nodes_.push_back(SnakeNode(sf::Vector2f(nodes_[nodes_.size() - 1].getPosition().x - 2 * SnakeNode::Radius,
                nodes_[nodes_.size() - 1].getPosition().y)));
            break;
        /*new*/
        case Direction::Any:
            nodes_.push_back(SnakeNode(sf::Vector2f(nodes_[nodes_.size() - 1].getPosition().x - 2 * SnakeNode::Radius * getDirectionVector().x,
                nodes_[nodes_.size() - 1].getPosition().y - 2 * SnakeNode::Radius * getDirectionVector().y)));
            break;
        }
    }
}
```

为Snake::move()添加了direction的状态为Any的情况：沿着directionVector的方向前进一个标准单位。

```
void Snake::move()
{
    for (decltype(nodes_.size()) i = nodes_.size() - 1; i > 0; --i)
    {
        nodes_[i].setPosition(nodes_.at(i - 1).getPosition());
    }

    switch (direction_)
    {
    case Direction::Up:
        nodes_[0].move(0, -2 * SnakeNode::Radius);
        break;
    case Direction::Down:
        nodes_[0].move(0, 2 * SnakeNode::Radius);
        break;
    case Direction::Left:
        nodes_[0].move(-2 * SnakeNode::Radius, 0);
        break;
    case Direction::Right:
        nodes_[0].move(2 * SnakeNode::Radius, 0);
        break;
    /*new*/
    case Direction::Any:
        nodes_[0].move(2 * SnakeNode::Radius * getDirectionVector().x,
            2 * SnakeNode::Radius * getDirectionVector().y);
        break;
    }
}
```

水果的控制

源代码只支持 1 种水果，颜色随机且贪食蛇增加的长度固定。现增加黑色、棕色、红、蓝色、绿色、 共5种水果，且贪食蛇吃了黑色、棕色水果不增加其长度，红色、蓝色、绿色水果增加的长度分别为 3、2、1；增加的长度在贪食蛇的尾部—假设初始是叠加在一起的。 系统随机生成上述5种水果，保持黑色和褐色水果所占比例为25%，其他的占75%。

具体改动：

为Fruit类增加了color_属性和getColor()成员函数。

```
class Fruit
{
public:
    Fruit(sf::Vector2f position, sf::Color color);

    void render(sf::RenderWindow& window);

    //sf::FloatRect getBounds() const;

    /*new*/
    sf::Color getColor() const;

    sf::Vector2f getPosition() const;

private:
    sf::CircleShape shape_;

    static const float Radius;

    /*new*/
    sf::Color color_;
};
```

重写了GameScreen::generateFruit()，更改了随机数种子，修改了随机数生成逻辑，使用多个随机数生成器，使得能在窗口中随机位置以25%和75%的比例随机生成黑色、褐色和红色、蓝色、绿色的水果。

```
void GameScreen::generateFruit()
/*new*/
{
    static std::default_random_engine engine;
    unsigned seed = std::chrono::system_clock::now().time_since_epoch().count();
    engine.seed(seed);
    static std::uniform_int_distribution<int> xDistribution(0, Game::Width - 2 * SnakeNode::Radius);
    static std::uniform_int_distribution<int> yDistribution(0, Game::Height - 2 * SnakeNode::Radius);
    static std::uniform_int_distribution<int> colorDistribution1(0, 3);
    static std::uniform_int_distribution<int> colorDistribution2RGB(0, 2);
    static std::uniform_int_distribution<int> colorDistribution2BlBr(0, 1);

    static std::vector<sf::Color> RGB{sf::Color(255, 0, 0), sf::Color(0, 255, 0), sf::Color(0, 0, 255)};
    static std::vector<sf::Color> BlBr{sf::Color(0, 0, 0), sf::Color(150, 75, 0)};
    if (colorDistribution1(engine))
    {
        fruit_.push_back(Fruit(sf::Vector2f(xDistribution(engine), yDistribution(engine)),
            RGB[colorDistribution2RGB(engine)]));
    }
    else
    {
        fruit_.push_back(Fruit(sf::Vector2f(xDistribution(engine), yDistribution(engine)),
            BlBr[colorDistribution2BlBr(engine)]));
    }
}
```

更改了Snake::checkFruitCollisions()函数，使其能够根据被吃掉的水果的颜色为蛇增加特定的长度。同时更改了Snake::grow()函数，为其增加了一个参数count，记录要为蛇增加的长度。

```
void Snake::checkFruitCollisions(std::vector<Fruit>& fruits)
{
    decltype(fruits.begin()) toRemove = fruits.end();

    for (auto it = fruits.begin(); it != fruits.end(); ++it)
    {
        /*new*/
        auto difference = it->getPosition() - nodes_[0].getPosition();
        float length = std::sqrt(std::pow(difference.x, 2) + std::pow(difference.y, 2));
        if (length <= SnakeNode::Radius * 2.20)
            toRemove = it;
        //if (it->getBounds().intersects(nodes_[0].getBounds()))
        //  toRemove = it;
    }

    if (toRemove != fruits.end())
    {
        pickupSound_.play();
        /*new*/
        auto color = toRemove->getColor();
        if (color == sf::Color::Red)
            grow(3);
        else if (color == sf::Color::Blue)
            grow(2);
        else if (color == sf::Color::Green)
            grow(1);
        fruits.erase(toRemove);
    }
}
```

绘制精灵版本的贪食蛇

源代码中的贪食蛇绘制过于简单—仅仅使用了矩形绘制。要求更改贪食蛇的绘制方法，头部使用图片，通过sprite进行绘制，其余部分使用园叠加黑色的矩形块绘制。

具体改动：

重写了Snake::render()函数，分别绘制蛇头和蛇身：

蛇头：根据direction_和directionVector计算蛇头Sprite图片的倾斜角度，使其正对着前进方向，并调用SnakeNode的renderH()成员函数。

蛇身：对每一节蛇身，根据它与前一节蛇身的相对位置计算蛇身Sprite图片的倾斜角度，使其正对着前一节蛇身的位置，并调用SnakeNode的renderB()成员函数。

```
/*new*/
void Snake::render(sf::RenderWindow& window)
{
    static double PI = 3.14159265358979323846;

    float rotationAngle;
    switch (direction_)
    {
    case Direction::Left:
        rotationAngle = -90;
        break;
    case Direction::Right:
        rotationAngle = 90;
        break;
    case Direction::Up:
        rotationAngle = 0;
        break;
    case Direction::Down:
        rotationAngle = 180;
        break;
    case Direction::Any:
        rotationAngle = atan2(directionVector_.y, directionVector_.x) * (180.0 / PI) + 90;
        break;
    }
    nodes_[0].renderH(window, rotationAngle);
    for (auto it = ++(nodes_.begin()); it != nodes_.end(); ++it)
    {
        auto difference = it -> getPosition() - (it-1) -> getPosition();
        float rotationAngleB = atan2(difference.y, difference.x) * (180.0 / PI) + 90;
        it->renderB(window, rotationAngleB);
    }
}
```

将SnakeNode类中原本的void render(sf::RenderWindow& window)成员函数扩写为void renderH(sf::RenderWindow& window, float rotationAngle)和void renderB(sf::RenderWindow& window, float rotationAngle)，分别绘制蛇头和蛇身的Sprite图片：

在SnakeNode::renderH()函数中，有static的sf::Sprite对象spriteHead，仅第一次调用时加载觅食蛇头部的图片文件，设置其中心和大小；后来每次调用函数时会重新更改spriteHead的旋转方向和位置，并最终绘制到窗口中。SnakeNode::renderB()函数同上。

```
/*new*/
void SnakeNode::renderH(sf::RenderWindow& window, float rotationAngle)
{
    static bool isInitialized = false;
    static sf::Texture textureHead;
    static sf::Sprite spriteHead;

    if (!isInitialized)
    {
        textureHead.loadFromFile("pics/snakehead.png");

        spriteHead.setTexture(textureHead);
        spriteHead.setOrigin(spriteHead.getGlobalBounds().width / 2, spriteHead.getGlobalBounds().height / 2


        spriteHead.setScale((2.2 * Radius) / spriteHead.getGlobalBounds().width * 1.3,
            (2.2 * Radius) / spriteHead.getGlobalBounds().height * 1.3);

        isInitialized = true;
    }

    spriteHead.setRotation(rotationAngle);

    spriteHead.setPosition(getPosition());

    window.draw(spriteHead);
}
```

```
/*new*/
void SnakeNode::renderB(sf::RenderWindow& window, float rotationAngle)
{
    static bool isInitialized = false;
    static sf::Texture textureBody;
    static sf::Sprite spriteBody;

    if (!isInitialized)
    {
        textureBody.loadFromFile("pics/snakebody.jpg");

        spriteBody.setTexture(textureBody);
        spriteBody.setOrigin(spriteBody.getGlobalBounds().width / 2, spriteBody.getGlobalBounds().height / 2);


        spriteBody.setScale((2.2 * Radius) / spriteBody.getGlobalBounds().width, (2.2 * Radius) / spriteBody.getGlobalBounds().height);

        isInitialized = true;
    }

    spriteBody.setRotation(rotationAngle);

    spriteBody.setPosition(getPosition());

    window.draw(spriteBody);
}
```

整体界面的修改

可以修改背景的颜色（提供白色、黑色、褐色三种）；允许显示（或关闭显示）网格，网格的颜色可以 设置（提供白色、黑色、褐色三种）。

具体改动：

将网格线相关的代码封装为Grid类，共有三个属性：

static const int cellSize_ = 16;　　//网格的单元格边长

sf::VertexArray gridLines_;　　//待绘制的网格，可直接传递给window的draw()函数

sf::Color gridColor_;　　//网格的颜色

和三个成员函数：

Grid(sf::Color gridColor);

初始化gridColor，创建gridLines对象并填充颜色。

void changeGridColor(sf::Color gridColor);

更新gridColor，并将gridLines的颜色更改为新的gridColor_。

void drawGridLines(sf::RenderWindow& window) const;

将gridLines_绘制到window上。

```
/*Grid.cpp*/

#include <SFML/Graphics.hpp>

#include "Grid.h"
#include "Game.h"

using namespace sfSnake;

Grid::Grid(sf::Color gridColor) : gridColor_(gridColor)
{

        const int numCellsX = static_cast<int>(1.0f * Game::Width / cellSize_);
        const int numCellsY = static_cast<int>(1.0f * Game::Height / cellSize_);


        gridLines_ = sf::VertexArray(sf::Lines, (numCellsX + numCellsY) * 2);


        for (int i = 0; i < numCellsY; ++i)
        {
            gridLines_[(i * 2)].position = sf::Vector2f(0, i * cellSize_);
            gridLines_[(i * 2) + 1].position = sf::Vector2f(Game::Width, i *
cellSize_);
            gridLines_[(i * 2)].color = gridColor_;
            gridLines_[(i * 2) + 1].color = gridColor_;
        }

        for (int i = 0; i < numCellsX; ++i)
        {
            gridLines_[(numCellsY * 2) + (i * 2)].position = sf::Vector2f(i *
cellSize_, 0);
            gridLines_[(numCellsY * 2) + (i * 2) + 1].position = sf::Vector2f(i *
cellSize_, Game::Height);
            gridLines_[(numCellsY * 2) + (i * 2)].color = gridColor_;
            gridLines_[(numCellsY * 2) + (i * 2) + 1].color = gridColor_;
        }
}

void Grid::drawGridLines(sf::RenderWindow& window) const
{
    window.draw(gridLines_);
}

void Grid::changeGridColor(sf::Color gridColor)
{
    if (gridColor_ != gridColor)
    {
        gridColor_ = gridColor;
```

```
        for (std::size_t i = 0; i < gridLines_.getVertexCount(); ++i)
            gridLines_[i].color = gridColor_;
    }
}
```

```
/*Grid.h*/

#ifndef GRID_H
#define GRID_H

#include <SFML/Graphics.hpp>

namespace sfSnake
{
class Grid
{
public:
    Grid(sf::Color gridColor);
    void changeGridColor(sf::Color gridColor);
    void drawGridLines(sf::RenderWindow& window) const;

private:
    static const int cellSize_ = 16;
    sf::VertexArray gridLines_;
    sf::Color gridColor_;
};
}


#endif
```

为Game类增加了bgColor_属性，初始设置为褐色sf::Color(150, 75, 0)。

```
/*new*/
sf::Color bgColor_ = sf::Color(150, 75, 0);
```

在Game::handleInput()中增加了处理键盘输入、改变背景颜色的过程，在游戏窗口打开时的任何时候都可以通过A、S、D来改变背景颜色。

```
/*new*/
else if (event.type == sf::Event::KeyPressed)
{
    if (event.key.code == sf::Keyboard::A)
        bgColor_ = sf::Color::White;
    else if (event.key.code == sf::Keyboard::S)
        bgColor_ = sf::Color::Black;
    else if (event.key.code == sf::Keyboard::D)
        bgColor_ = sf::Color(150, 75, 0);
```

在Game::render()中通过window.clear(bgColor)实时更新背景颜色。

```
void Game::render()
{
    /*new*/
    window_.clear(bgColor_);
    Game::Screen->render(window_);
    window_.display();
}
```

在GameScreen类中增加了bool gridOn，Grid grid_两个属性，作用分别是表示网格是否开启和创建Grid实例。

```
/*new*/
if (sf::Keyboard::isKeyPressed(sf::Keyboard::G))
    gridOn_ = !gridOn_;
```

```
void GameScreen::render(sf::RenderWindow& window)
{
    /*new*/
    if (gridOn_)
        grid_.drawGridLines(window);
```

在GameScreen类中增加了sf::Font font，sf::Text text两个属性，用于显示说明文本。

```
GameScreen::GameScreen() : snake_(), grid_(sf::Color::Black)
{
    /*new*/
    font_.loadFromFile("Fonts/game_over.ttf");
    text_.setFont(font_);
```

```
/*new*/
window.draw(text_);
```

在构造函数GameScreen::GameScreen()中用黑色初始化了grid实例，并加载了用于显示文本的font和text对象。

```
GameScreen::GameScreen() : snake_(), grid_(sf::Color::Black)
{
    /*new*/
    font_.loadFromFile("Fonts/game_over.ttf");
    text_.setFont(font_);
    text_.setString(
        "Press [A][S][D] to select the background color: white, black, brown"
        "\n\nPress [G] to show the grid"
        "\n\nPress [J][K][L] to select the grid color: white, black, brown");
    text_.setFillColor(sf::Color::White);
    text_.setCharacterSize(15);

    sf::FloatRect textBounds = text_.getLocalBounds();
    text_.setOrigin(textBounds.left + textBounds.width, textBounds.top);
    text_.setPosition(Game::Width - 10, 10);
}
```

在GameScreen::handleInput()中增加了处理键盘输入、打开/关闭网格和改变网格颜色的过程，在游戏进行中（即Game::Screen指向一个GameScreen对象时）可以随时通过G打开/关闭网格，通过J、K、L改变网格颜色。

```
void GameScreen::handleInput(sf::RenderWindow& window)
{
    /*new*/
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::G))
        gridOn_ = !gridOn_;

    if (sf::Keyboard::isKeyPressed(sf::Keyboard::J))
        grid_.changeGridColor(sf::Color::White);
    else if (sf::Keyboard::isKeyPressed(sf::Keyboard::K))
        grid_.changeGridColor(sf::Color::Black);
    else if (sf::Keyboard::isKeyPressed(sf::Keyboard::L))
        grid_.changeGridColor(sf::Color(150, 75, 0));


    snake_.handleInput(window);
}
```

在GameScreen::render()，新增了绘制网格和绘制说明文本的代码。

```
void GameScreen::render(sf::RenderWindow& window)
{
    /*new*/
    if (gridOn_)
        grid_.drawGridLines(window);

    /*new*/
    window.draw(text_);


    snake_.render(window);

    for (auto fruit : fruit_)
        fruit.render(window);
}
```

理清代码

代码中，要仔细考虑水果、蛇(蛇头、其他节点)、网格等对象的生命周期，确保你设计的对象周期模型是经济可靠的。

这里不做赘述。

漏洞修复

（1） 将旧的setColor()函数更改为了新标准中的setFillColor()函数。

```
MenuScreen::MenuScreen()
{
    font_.loadFromFile("Fonts/game_over.ttf");
    text_.setFont(font_);
    text_.setString(
        "\n\n\n\n\n\n\n\n\nPress [SPACE] to play"
        "\n\nPress [ESC] to quit");

    snakeText_.setFont(font_);
    snakeText_.setString("Snake!");
    snakeText_.setFillColor(sf::Color::Green);
    snakeText_.setCharacterSize(64);
    snakeText_.setStyle(sf::Text::Bold);
```

（2） 在GameScreen::generateFruit()中更改了seed更新逻辑，避免了水果刷新在同一行的问题。借助库，使用当前高精度时间std::chrono::system_clock::now().time_since_epoch().count()作为新的种子。

```
void GameScreen::generateFruit()
/*new*/
{
    static std::default_random_engine engine;
    unsigned seed = std::chrono::system_clock::now().time_since_epoch().count();
    engine.seed(seed);
    static std::uniform_int_distribution<int> xDistribution(0, Game::Width - 2 * SnakeNode::Radius);
    static std::uniform_int_distribution<int> yDistribution(0, Game::Height - 2 * SnakeNode::Radius);
    static std::uniform_int_distribution<int> colorDistribution1(0, 3);
    static std::uniform_int_distribution<int> colorDistribution2RGB(0, 2);
    static std::uniform_int_distribution<int> colorDistribution2BlBr(0, 1);

    static std::vector<sf::Color> RGB{sf::Color(255, 0, 0), sf::Color(0, 255, 0), sf::Color(0, 0, 255)};
    static std::vector<sf::Color> BlBr{sf::Color(0, 0, 0), sf::Color(150, 75, 0)};
    if (colorDistribution1(engine))
    {
        fruit_.push_back(Fruit(sf::Vector2f(xDistribution(engine), yDistribution(engine)),
            RGB[colorDistribution2RGB(engine)]));
    }
    else
    {
        fruit_.push_back(Fruit(sf::Vector2f(xDistribution(engine), yDistribution(engine)),
            BlBr[colorDistribution2BlBr(engine)]));
    }
}
```

程序运行逻辑改进

（1） 在Snake类的handleInput()成员函数中，增加一条逻辑：

当用键盘上的↑←↓→键控制方向时，如果输入的方向与当前觅食蛇的前进方向相反，不会改变觅食蛇的前进方向，避免觅食蛇直接撞到自己身体而导致游戏结束。

```
if (sf::Keyboard::isKeyPressed(sf::Keyboard::Up) && direction_ != Direction::Down)
    direction_ = Direction::Up;
else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Down) && direction_ != Direction::Up)
    direction_ = Direction::Down;
else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Left) && direction_ != Direction::Right)
    direction_ = Direction::Left;
else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Right) && direction_ != Direction::Left)
    direction_ = Direction::Right;
```

（2） 将蛇身改为圆形状，改变了蛇头与自身以及蛇头与水果的碰撞逻辑：

为Fruit类增加了getPosition()成员函数。重写了Snake::checkFruitCollisions()函数，判断蛇吃到水果的逻辑变为水果和蛇头的中心距离是否小于等于蛇身半径的2.20倍。

```
/*new*/
auto difference = it->getPosition() - nodes_[0].getPosition();
float length = std::sqrt(std::pow(difference.x, 2) + std::pow(difference.y, 2));
if (length <= SnakeNode::Radius * 2.20)
    toRemove = it;
```

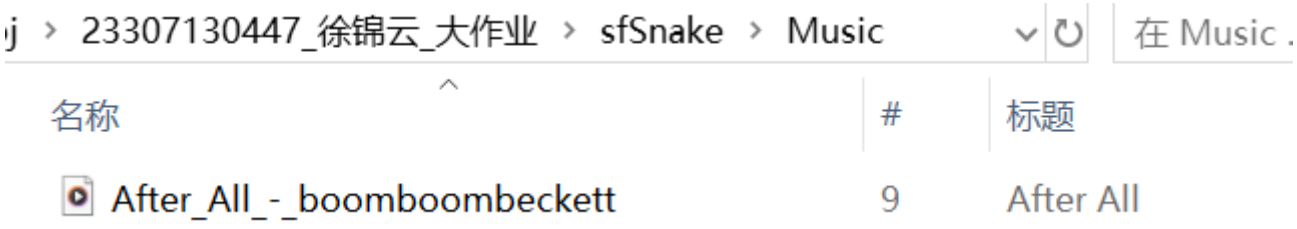
重写了Snake::checkSelfCollisions()函数，判断蛇与自身碰撞的逻辑变为距离是否小于等于蛇身半径的1.90倍。（这是为了降低难度）

```
void Snake::checkSelfCollisions()
{
    SnakeNode& headNode = nodes_[0];

    /*new*/
    for (decltype(nodes_.size()) i = 1; i < nodes_.size(); ++i)
    {
        auto difference = headNode.getPosition() - nodes_[i].getPosition();
        float length = std::sqrt(std::pow(difference.x, 2) + std::pow(difference.y, 2));
        if (length <= SnakeNode::Radius * 1.90)
        {
            dieSound_.play();
            sf::sleep(sf::seconds(dieBuffer_.getDuration().asSeconds()));
            hitSelf_ = true;
            break;
        }
    }
}
```

视听效果修改

（1） 更改了背景音乐为：After All

下载途径合法，网站链接：https://www.jamendo.com/start



（2） 原来的窗口太小，因此在Game类中将窗口的长和宽增加为原来的两倍：

```
static const int Width = 640 * 2;
static const int Height = 480 * 2;
```

其他的可行性修改

该部分代码是提升游戏体验的一种设想，没有体现在源代码，如果综合下面的逻辑，某些程度上会造成功能的重复，只是在文件存储大小、代码结构优化上有一定优势，在此会做简单的说明。

关于水果的分布

通过随机数分布实现颜色多样化，不同颜色对应不同生成概率，增加游戏策略性（例如绿色水果更容易出现）。

```
//example
// 新增颜色概率控制（黑色12.5%、棕色12.5%、红色12.5%、蓝色25%、绿色37.5%）
void Fruit::judgeColor(){
    switch(generateRandom()){
        case 0: color_=sf::Color::Black; break;
        case 1: color_=sf::Color{165,42,42}; break; // 棕色
        case 2: color_=sf::Color::Red; break;
        case 3: case 4: color_=sf::Color::Blue; break; // 蓝色占25%
        default: color_=sf::Color::Green; // 绿色占37.5%
    }
}//在别的函数处增加概率控制逻辑，可以提升玩家的策略性选择
```

在项目中，每次小蛇出现，界面中都只有一个随机生成的水果。如果一开始是全屏随机生成，随着蛇的长度增加，水果生成范围逐渐缩小（从全屏→内缩10像素→内缩20像素），增加游戏后期难度，那么，这就比较贴近原始游戏的效果。

```
// 根据蛇的长度分阶段生成水果
void GameScreen::update(sf::Time delta) {
    if (fruit_.size() == 0) {
        if (snake_.getSize() < 15) generateFruit1();  // 全屏生成
        else if (snake_.getSize() < 20) generateFruit2(); // 范围缩小10像素
        else generateFruit3(); // 范围缩小20像素
    }
}
```

关于蛇的身体构造

在游戏中，我将蛇的身体使用的圆形+矩形装饰作为一个完整的模块（也就是snakebody.jpg）来进行增长，这类似于一个简化的处理，视觉效果和直接绘制是一样的，但是如果不做模块化处理，那么可以大致像下面这样操作：

```
// 蛇头使用纹理图片，身体使用圆形+矩形装饰
SnakeNode::SnakeNode(sf::Vector2f position, bool isHead, float headSize) :
isHead_(isHead) {
    if (isHead) {
        headTexture_.loadFromFile("Picture/Head.png");
        headSprite_.setTexture(headTexture_);
        headSprite_.setScale(headSize / textureSize.x, headSize / textureSize.y);
    } else {
        shape_.setFillColor(sf::Color{255,192,203}); // 粉色身体
        rectangle_.setFillColor(sf::Color::White);   // 白色装饰线
    }
}
```

关于边界问题

分阶段边界规则：短蛇可穿越边界，长蛇触碰边界则死亡，增加游戏阶段性的挑战性。

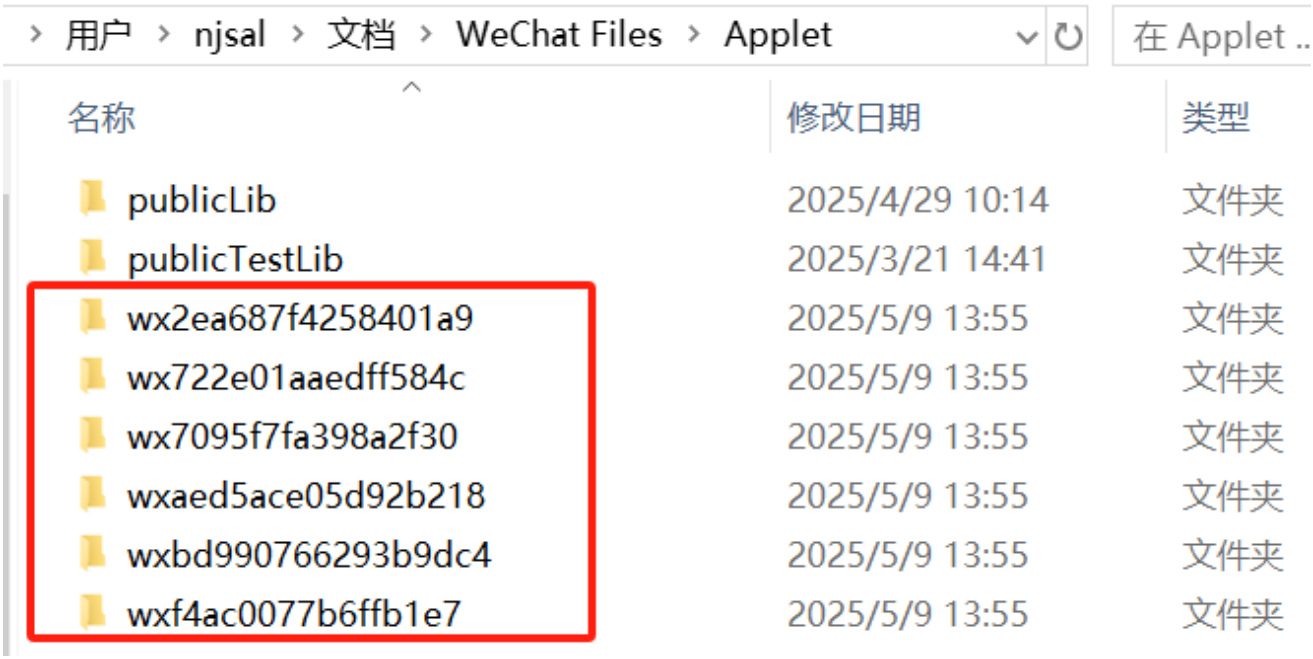
```
// 蛇较长时触碰边界直接死亡
void Snake::checkEdgeCollisions() {
    if (getSize() >= 15) { // 长度≥15时边界不可穿越
        if (headNode.getPosition().x <= 10.f || headNode.getPosition().x >=
Game::Width -10.f)
            hitSelf_ = true;
    } else { // 短蛇可穿越边界
        if (headNode.getPosition().x <= 0)
            headNode.setPosition(Game::Width, headNode.getPosition().y);
        // ...其他方向类似
    }
}
```

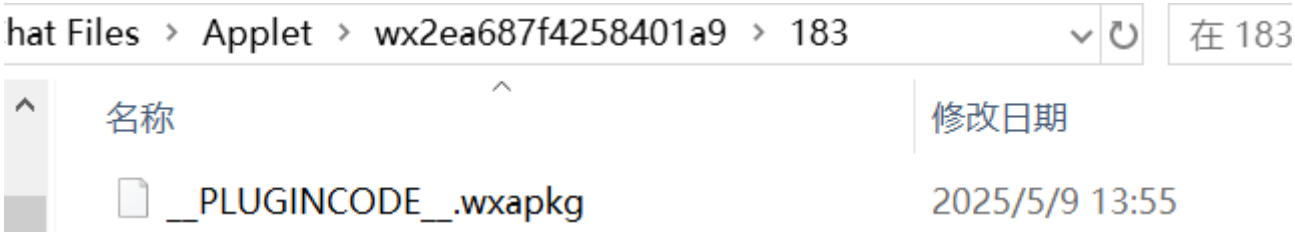
对于正规贪吃蛇游戏的一些分析以及对于本项目的启发

微信小程序上有正规的贪吃蛇游戏，且该项目代码主要由javascipt和json文件等构成，虽然文件经过加密且被分作几个子包，但是其加密程度不高，可以简单破解并分析。我们没有学过javascript语言，但是javascript在每一个类内部的语法与C/C++十分相似，所以权当学习资料也是可行的。

解密过程

首先打开小程序，使得wechat files获取相关的加密包：





这些文件夹内主要都是.wxapkj文件，这就是我们的目标文件。

经过解密，我们初步得到以下文件，这些文件是多个子包共同得到的：

wx2ea687f4258401a9.wxapkg	2025/5/9 14:03
wx722e01aaedff584c.wxapkg	2025/5/9 14:04
wx7095f7fa398a2f30.wxapkg	2025/5/9 14:04
wxaed5ace05d92b218.wxapkg	2025/5/9 14:04
wxbd990766293b9dc4.wxapkg	2025/5/9 14:04
wxf4ac0077b6ffb1e7.wxapkg	2025/5/9 14:05

通过对这些文件进行逆向编译。先安装一些依赖：

```
npm install
npm install esprima
npm install css-tree
npm install cssbeautify
npm install vm2
npm install uglify-es
npm install js-beautify
```

正式逆向：

```
.\bingo.bat [路径]
```

得到类似这种：

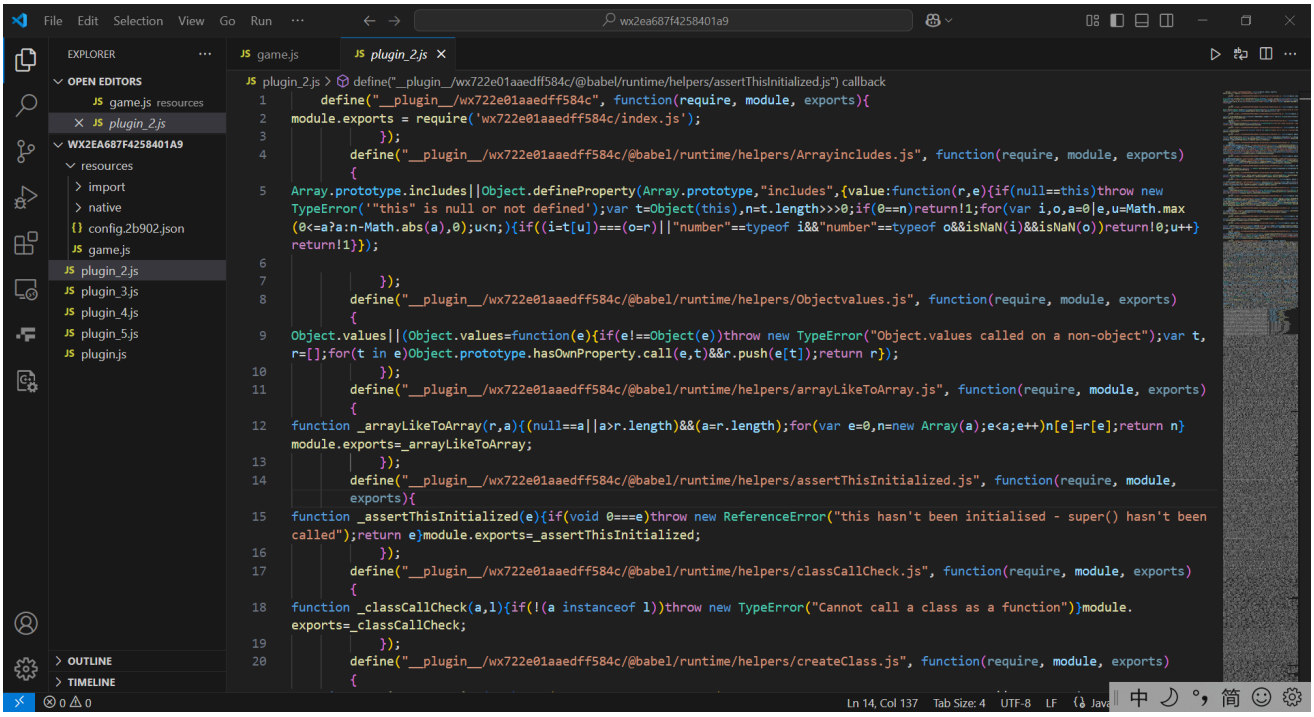
```
Header info:
  firstMark: 0xbe
  unknownInfo:  0
  infoListLength:  50
  dataLength:  211827
  lastMark: 0xed

File list info:
  fileCount:  2
Saving files...
Unpack done.
(                                                                    \wuWxapkg.js:176
            throw new Error("检测到此包是分包后的子包，请通过 -s 参数指定存放路径后重试，如 node wuWxapkg.js -s=/xxx/xxx
 ./testpkg/test-pkg-sub.wxapkg");
            ^

Error: 检测到此包是分包后的子包，请通过 -s 参数指定存放路径后重试，如 node wuWxapkg.js -s=/xxx/xxx ./testpkg/test-pkg-su
b.wxapkg
    at Array.packDone (C:\360Downloads\                                    \wuWxapkg.js:176:19)
    at CntEvent.decount (C:\360Downloads\                                  :\wuLib.js:22:54)
    at C:\360Downloads\                                                    \wuLib.js:95:17
    at agent (C:\360Downloads\                                             \wuLib.js:66:23)
    at FSReqCallback.oncomplete (node:fs:191:23)

Node.js v20.11.1

(                                                                    >goto:eof
```

依次解密，得：

名称	修改日期	类型
wx2ea687f4258401a9	2025/5/9 14:15	文件夹
wx722e01aaedff584c	2025/5/9 14:20	文件夹
wx7095f7fa398a2f30	2025/5/9 14:21	文件夹
wxaed5ace05d92b218	2025/5/9 14:21	文件夹
wxbd990766293b9dc4	2025/5/9 14:22	文件夹
wxf4ac0077b6ffb1e7	2025/5/9 14:22	文件夹

在文件夹内部查看javascript代码即可获取大致逻辑。

正式开始分析



上面就是代码的大致框架，在这一部分，我不会聚焦于贪吃蛇功能的实现，而是希望探索在实际的大型项目中，当功能繁杂、插件很多时，如何有效的管理前端后端，真正的探索“面向对象”意义所在。上述代码的可取之处在于：

1. 模块化与动态加载

- **动态插件加载机制**：
  `global.requirePlugin` 函数通过别名动态解析插件路径，并调用 `require` 加载目标模块。这种设计支持按需加载插件，减少初始资源消耗，提升灵活性。
- **模块化定义**：
  使用 `define` 和 `require` 管理模块依赖（类似 AMD/CommonJS），明确划分功能边界，避免全局变量污染，便于代码维护和复用。

2. 现代语法转译支持

- **Babel 运行时辅助函数**：
  代码中包含了 `@babel/runtime` 的辅助函数（如 `_createClass` 、 `_inherits` ），表明项目通过 Babel 将 ES6+ 的类、继承等语法转译为 ES5。这使得开发者可以编写现代 JavaScript，同时兼容旧环境，提升代码可读性和可维护性。

3. 兼容性处理

- **原型方法补丁**：
  为不支持 `Array.prototype.includes` 和 `Object.values` 的环境添加了兼容实现（通过 `Object.defineProperty` ），确保代码在低版本浏览器或特殊环境中正常运行。
- **迭代器辅助函数**：
  `_createForOfIteratorHelper` 等函数处理了旧环境中 `for...of` 循环的兼容性问题，增强代码鲁棒性。

4. 依赖管理

- **插件映射与别名系统**：
  `pluginInfoMap` 管理插件别名和实际路径的映射，简化了插件引用路径，降低耦合度。例如，通过别名 `wx722e01aaedff584c` 快速定位到具体插件模块。

5. 代码组织

- **分模块实现工具函数**：
  将 Babel 辅助函数、兼容性补丁等功能拆分为独立模块（如 `defineProperty.js` 、 `classCallCheck.js` ），符合单一职责原则，便于复用和调试。

效果展示

编译环境说明(5);界面展示(5);运行界面展示及描述(10)；视频链接(5)。

关于cmake

```
cmake_minimum_required(VERSION 3.16)
project(CMakeSFMLProject LANGUAGES CXX)

set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)

set(SFML_DIR "C:\\360Downloads\\SFML-2.6.2\\lib\\cmake\\SFML")
find_package(SFML 2.6 COMPONENTS graphics audio system window network REQUIRED)

set(EXECUTABLE_NAME sfSnake)
set(SOURCE_FILES
    sfSnake-master/main.cpp
    sfSnake-master/Game.cpp
    sfSnake-master/Screen.cpp
    sfSnake-master/Fruit.cpp
    sfSnake-master/GameOverScreen.cpp
    sfSnake-master/GameScreen.cpp
    sfSnake-master/MenuScreen.cpp
    sfSnake-master/Snake.cpp
    sfSnake-master/SnakeNode.cpp
    sfSnake-master/Grid.cpp
)

add_executable(${EXECUTABLE_NAME} ${SOURCE_FILES})

target_link_libraries(${EXECUTABLE_NAME} PRIVATE
    sfml-graphics
    sfml-audio
    sfml-system
```

```
        sfml-window
        sfml-network
    )

    target_compile_features(${EXECUTABLE_NAME} PRIVATE cxx_std_17)

    file(COPY
        Fonts/
        Music/
        pics/
        Sounds/
        DESTINATION ${CMAKE_RUNTIME_OUTPUT_DIRECTORY}/Debug
    )

    add_custom_command(TARGET ${EXECUTABLE_NAME} POST_BUILD
        COMMAND ${CMAKE_COMMAND} -E copy_directory
        "C:\\360Downloads\\SFML-2.6.2\\bin"
        $<TARGET_FILE_DIR:${EXECUTABLE_NAME}>
    )
```

用 `find_package` 直接链接本地安装的 SFML。

`file(COPY ...)` ：将资源文件（字体、图片、音频）复制到可执行文件目录，避免运行时因路径错误找不到资源。

SFML 需要动态链接库（DLL）才能运行。即使编译成功，若未正确配置 DLL，程序会静默退出。因此最后一段代码就是将dll文件直接复制进bin\debug目录的。

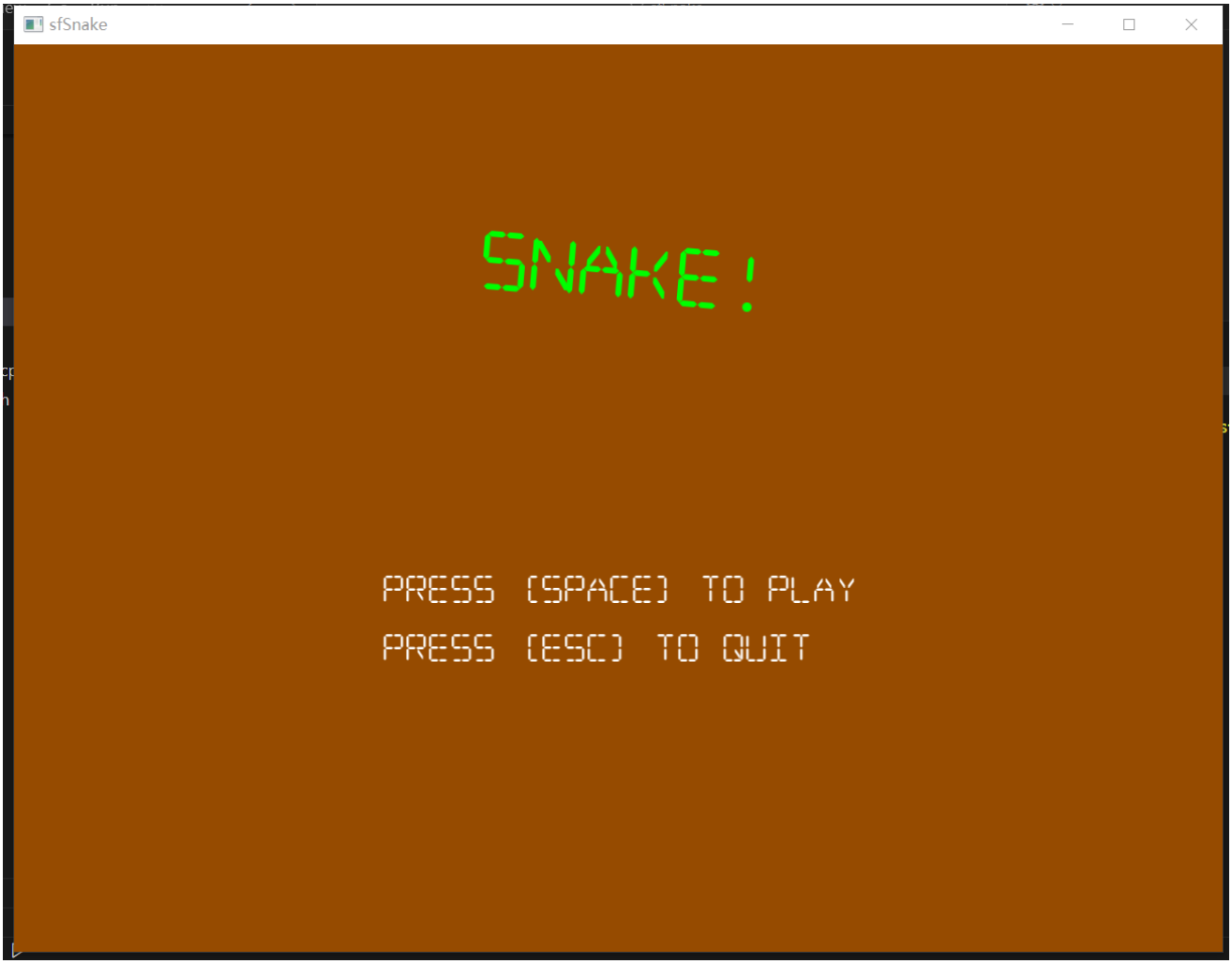
执行

```
# 清理旧构建（确保重新生成）
rm -r build

# 生成并编译
cmake -S . -B build
cmake --build build

# 运行测试程序
.\build\bin\Debug\sfSnake.exe
```

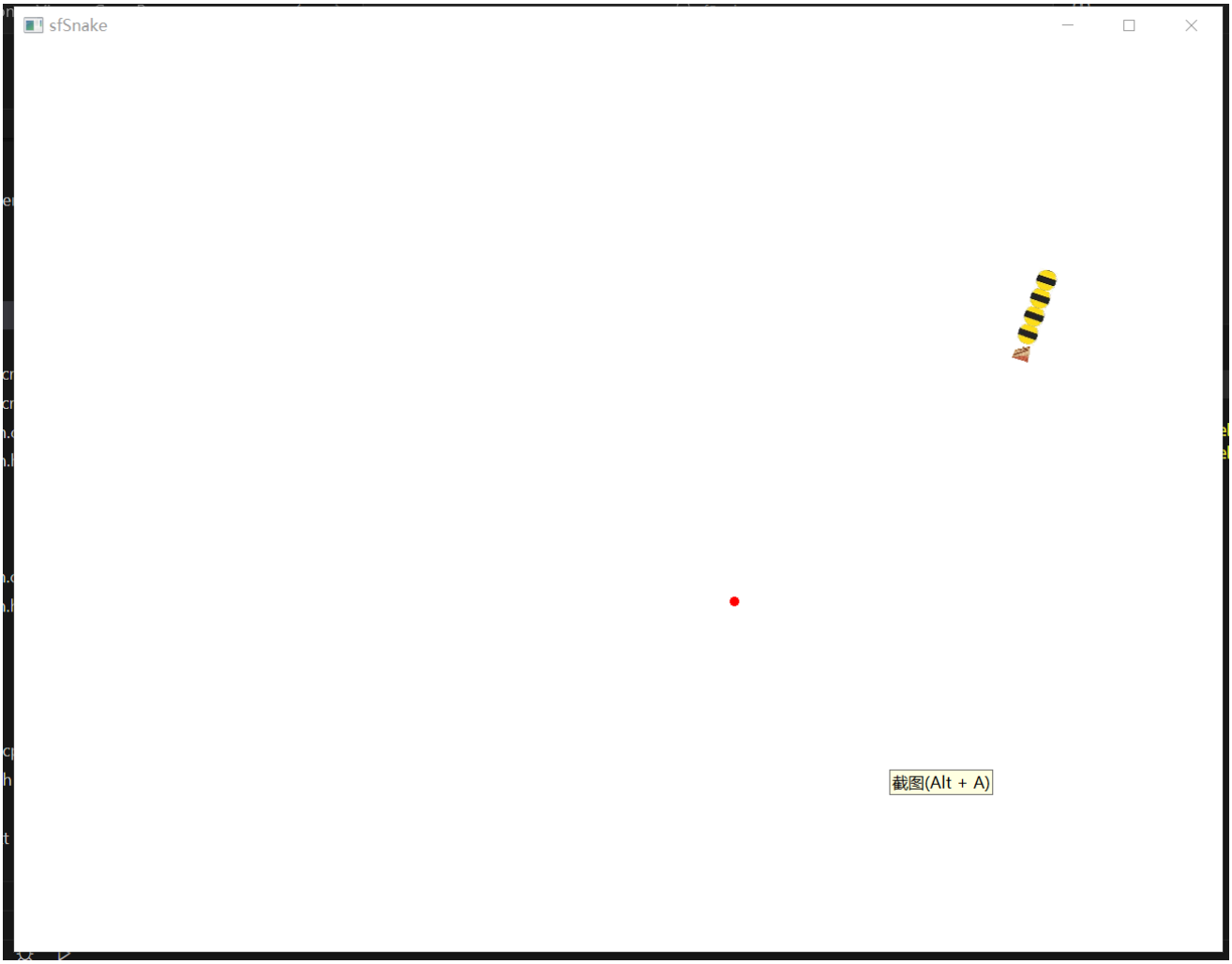
```
PS C:\Users\njsal\Desktop\复旦大学\41面向对象程序设计\lab\pj\23307130447_徐锦云_大作业\sfSnake> cmake -S . -B build
-- Building for: Visual Studio 17 2022
-- Selecting Windows SDK version 10.0.22621.0 to target Windows 10.0.19045.
-- The CXX compiler identification is MSVC 19.37.32824.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: C:/Program Files/Microsoft Visual Studio/2022/Community/VC/Tools/MSVC/14.37.32822/bin/Hostx64/x64/cl.
exe - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Found SFML 2.6.2 in C:/360Downloads/SFML-2.6.2/lib/cmake/SFML
-- Configuring done (9.4s)
-- Generating done (0.1s)
-- Build files have been written to: C:/Users/njsal/Desktop/复旦大学/41面向对象程序设计/lab/pj/23307130447_徐锦云_大作业/sfSnake/build
PS C:\Users\njsal\Desktop\复旦大学\41面向对象程序设计\lab\pj\23307130447_徐锦云_大作业\sfSnake> cmake --build build
适用于 .NET Framework MSBuild 版本 17.7.2+d6990bcfa

  1>Checking Build System
  Building Custom Rule C:/Users/njsal/Desktop/复旦大学/41面向对象程序设计/lab/pj/23307130447_徐锦云_大作业/sfSnake/CMakeLists.txt
  main.cpp
  Game.cpp
  Screen.cpp
  Fruit.cpp
  GameOverScreen.cpp
  GameScreen.cpp
  MenuScreen.cpp
  Snake.cpp
  SnakeNode.cpp
  Grid.cpp
  正在生成代码...
  sfSnake.vcxproj -> C:\Users\njsal\Desktop\复旦大学\41面向对象程序设计\lab\pj\23307130447_徐锦云_大作业\sfSnake\build\bin\Debug\sfSnake
.exe
  Building Custom Rule C:/Users/njsal/Desktop/复旦大学/41面向对象程序设计/lab/pj/23307130447_徐锦云_大作业/sfSnake/CMakeLists.txt
PS C:\Users\njsal\Desktop\复旦大学\41面向对象程序设计\lab\pj\23307130447_徐锦云_大作业\sfSnake> .\build\bin\Debug\sfSnake.exe
PS C:\Users\njsal\Desktop\复旦大学\41面向对象程序设计\lab\pj\23307130447_徐锦云_大作业\sfSnake>
```

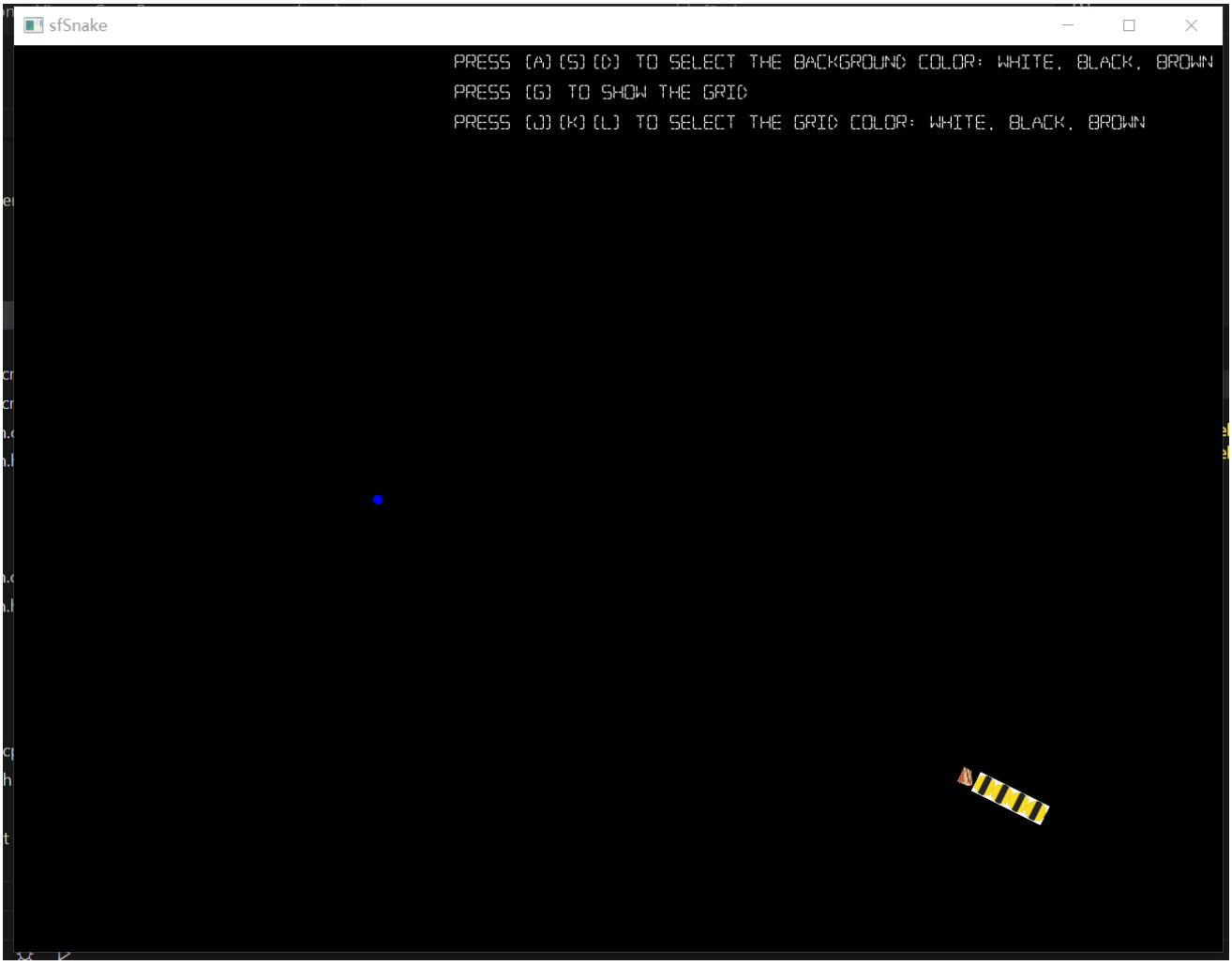
运行界面

首页如下：

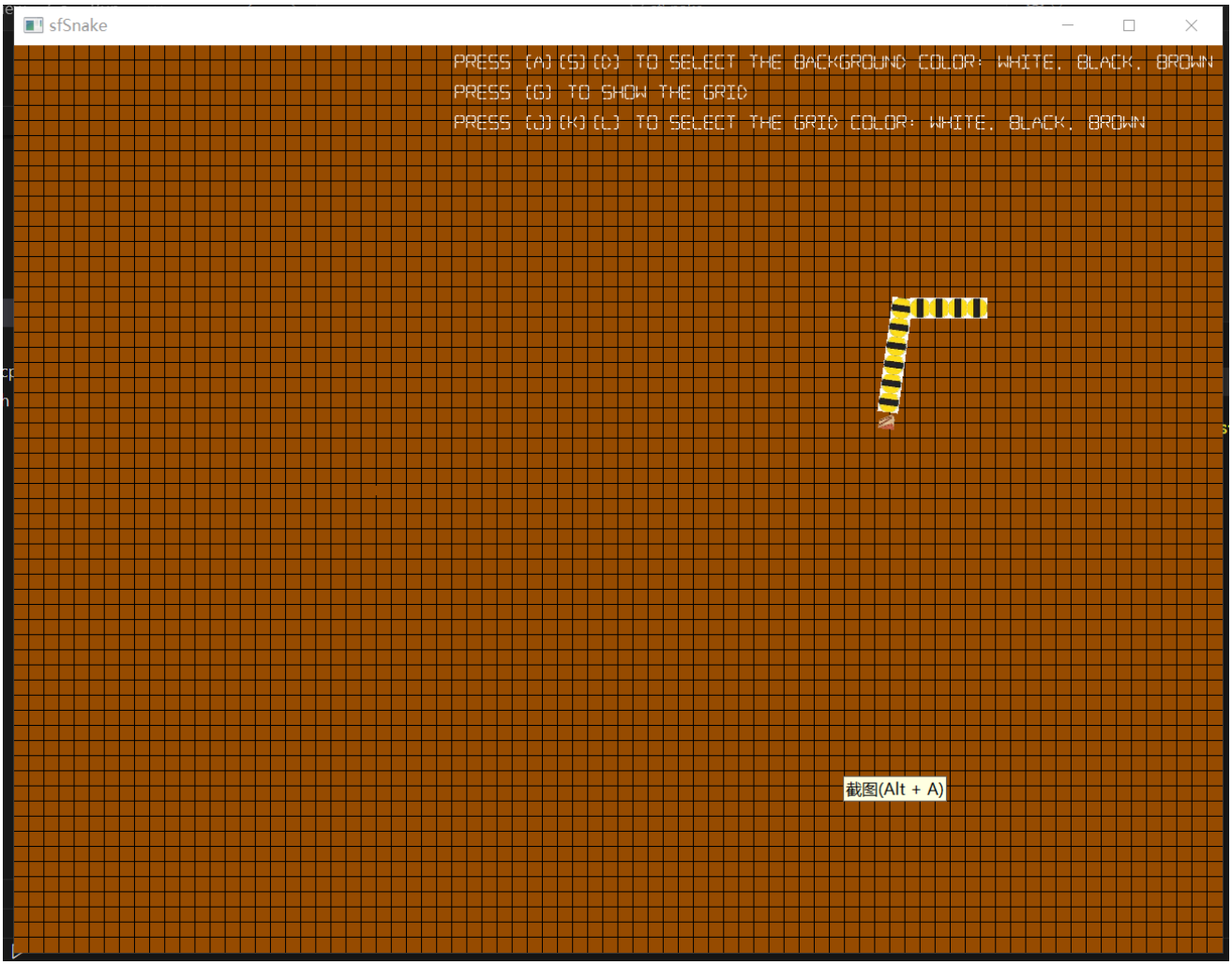


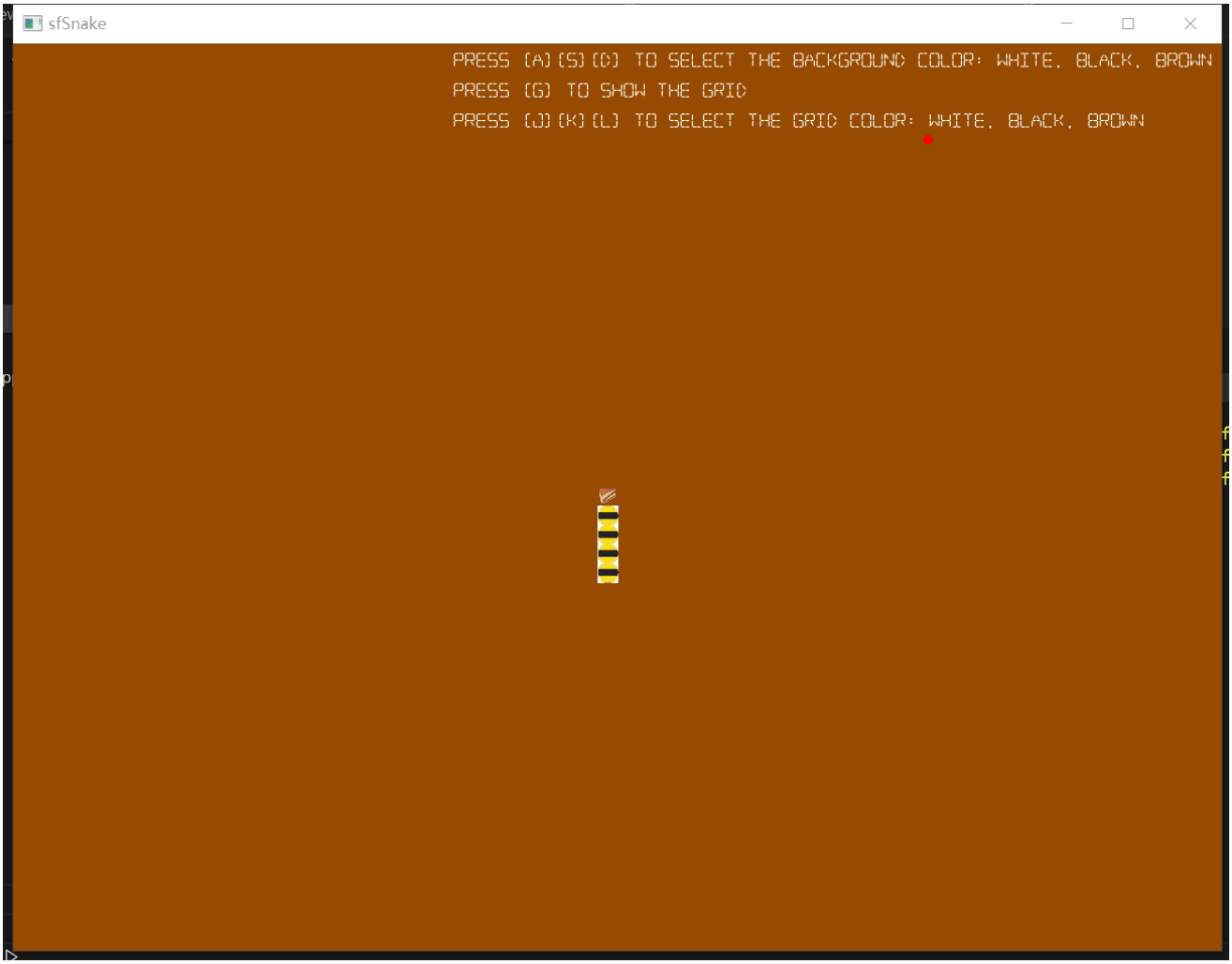
开始游戏：按不同按键会有不同的背景颜色，也会有不同的水果，可以通过上下左右四个箭头和鼠标来控制移动，以及图中的按键如ASD来控制背景。

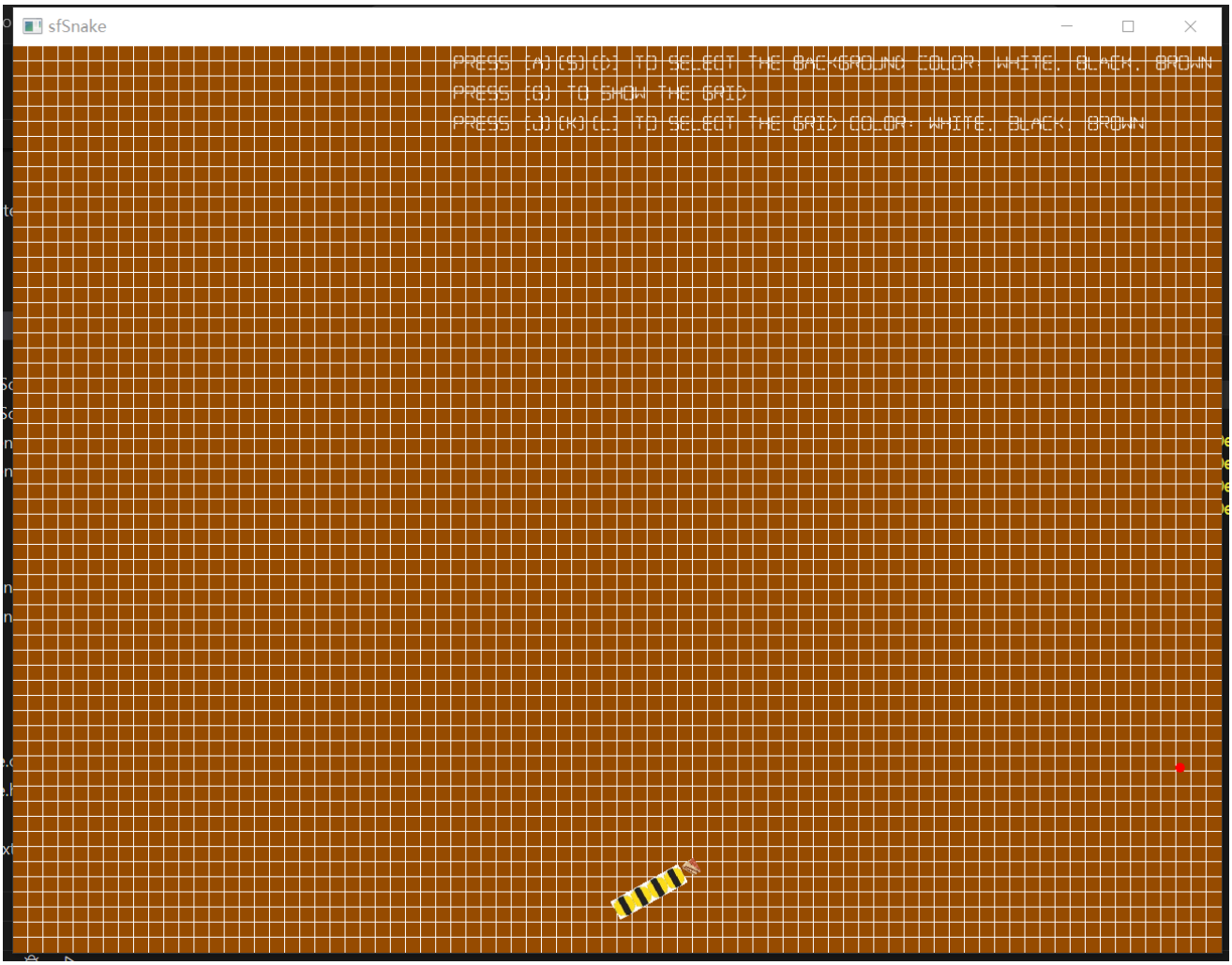

sfSnake
截图(Alt + A)


sfSnake
PRESS [A][S][D] TO SELECT THE BACKGROUND COLOR: WHITE, BLACK, BROWN
PRESS [G] TO SHOW THE GRID
PRESS [J][K][L] TO SELECT THE GRID COLOR: WHITE, BLACK, BROWN

关于网格线的展示，可以通过G控制是否有网格线，通过JKL来控制网格线的颜色：




sfSnake
PRESS [A][S][D] TO SELECT THE BACKGROUND COLOR: WHITE, BLACK, BROWN
PRESS [G] TO SHOW THE GRID
PRESS [J][K][L] TO SELECT THE GRID COLOR: WHITE, BLACK, BROWN


sfSnake
PRESS [A][S][D] TO SELECT THE BACKGROUND COLOR: WHITE, BLACK, BROWN
PRESS [G] TO SHOW THE GRID
PRESS [J][K][L] TO SELECT THE GRID COLOR: WHITE, BLACK, BROWN

结算画面：

视频

链接： https://www.bilibili.com/video/BV1qq55zsEiS/?share_source=copy_web&vd_source=7d71702f45b0399a539dacd94f06cb4e

具体内容见视频展示。